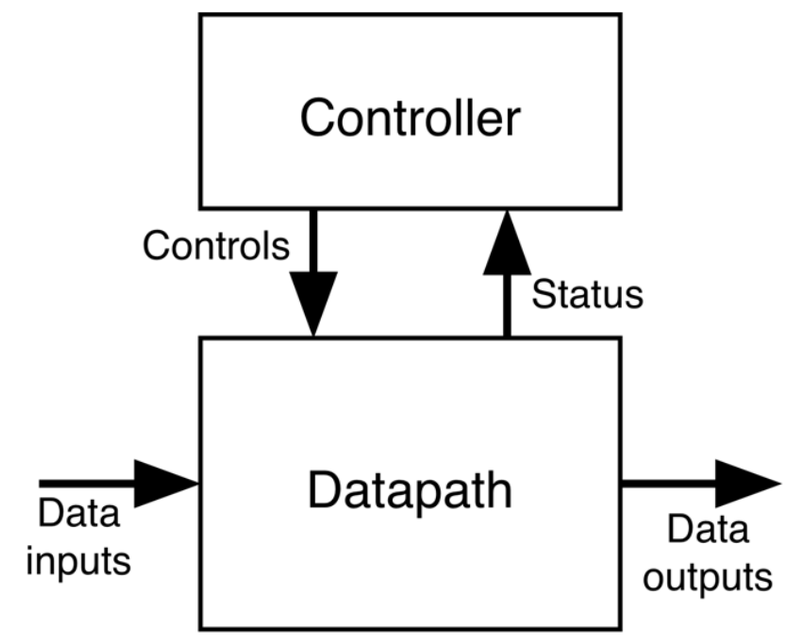
7A. STATE MACHINES

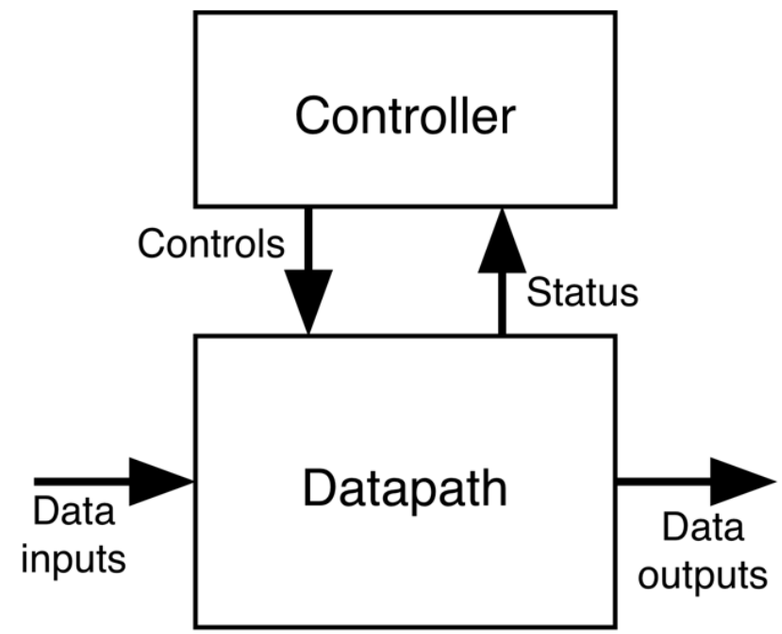
DATAPATH ARCHITECTURE

- The majority of circuits have it
- Circuit divided into controller & datapath
- Datapath contains all DSP
- Controller determines sequence of operations
- Communication between the two is tightly controlled
- Staus vs. control signals



DATAPATH ARCHITECTURE

- All data going from datapath to controller is status
- Controller uses status to decide current outputs and next state
- The state decides how the data path should be configured
- Through control signals

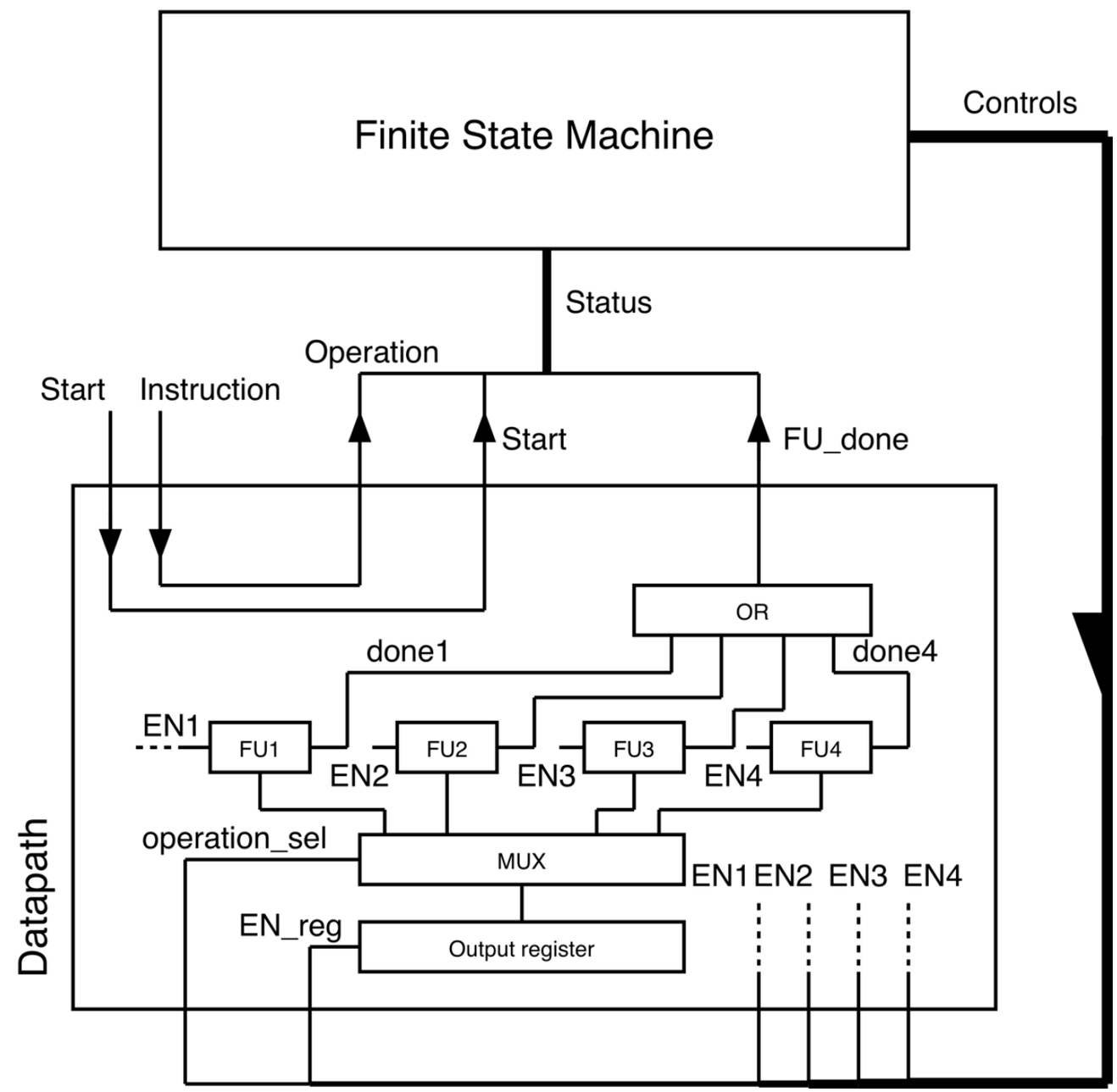


STRATEGY FOR DESIGNING STATE MACHINES

- Examine system
- Decide on minimum set of states
- Draw state diagram / write state table
- Use the three process approach to write HDL for the state machine

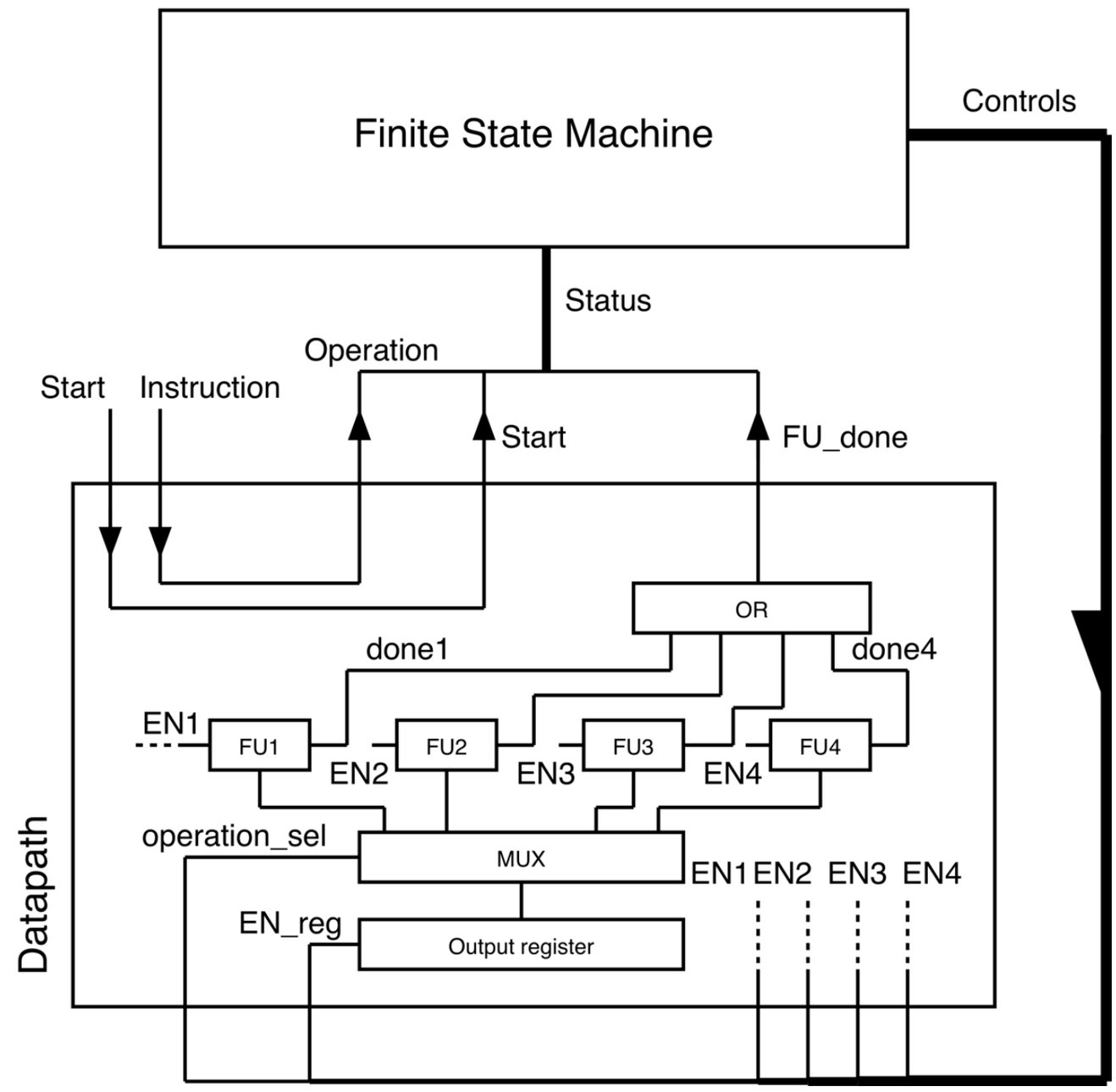
EXAMPLE

- Some understanding of the data path is necessary
- Understanding of controls and status a must



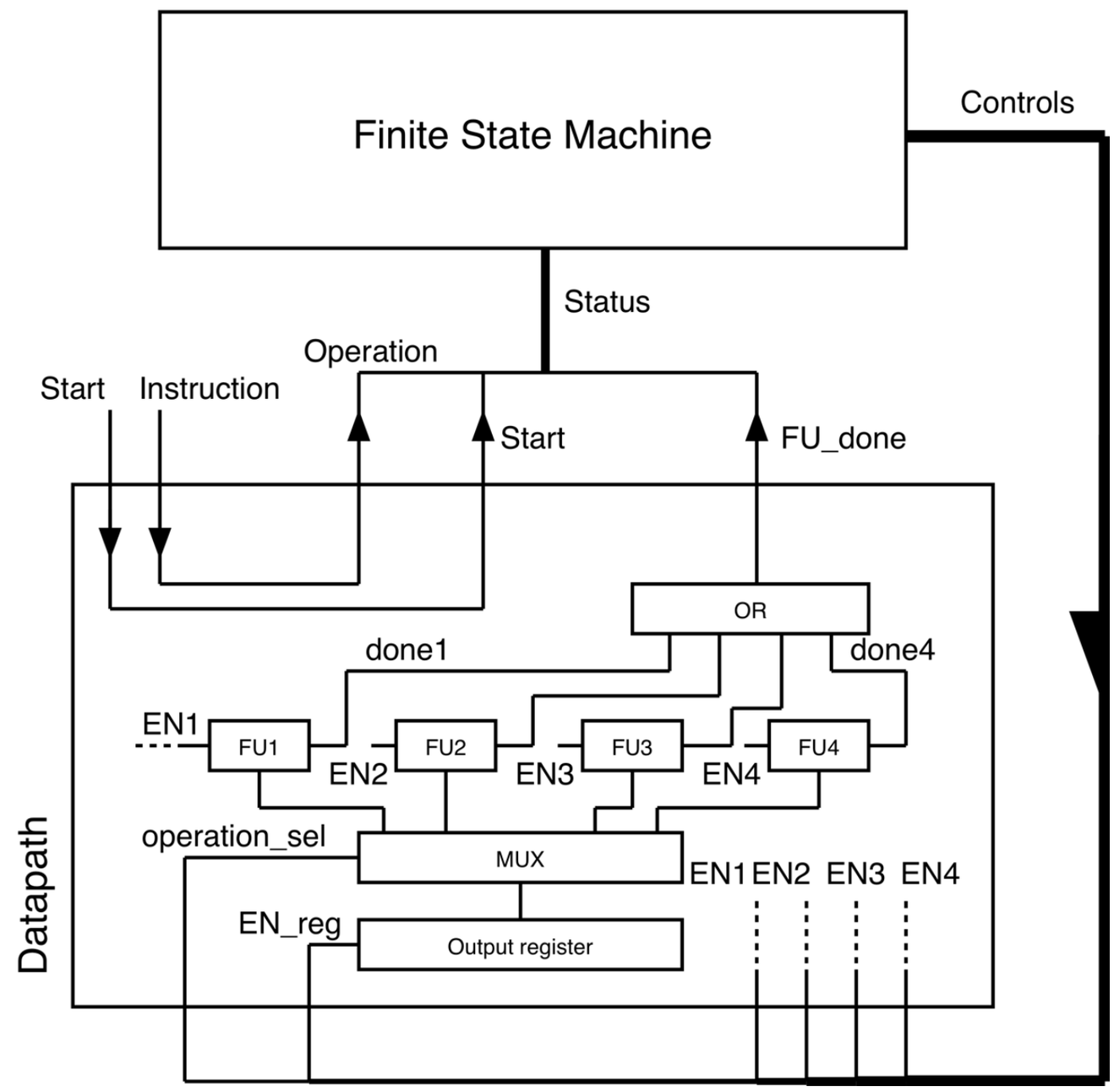
STATUS SIGNALS

- Operation
- Start
- FU_done



CONTROL SIGNALS

- En1, En2, En3, En4
- En_reg
- Operation_sel



PICK THE STATES

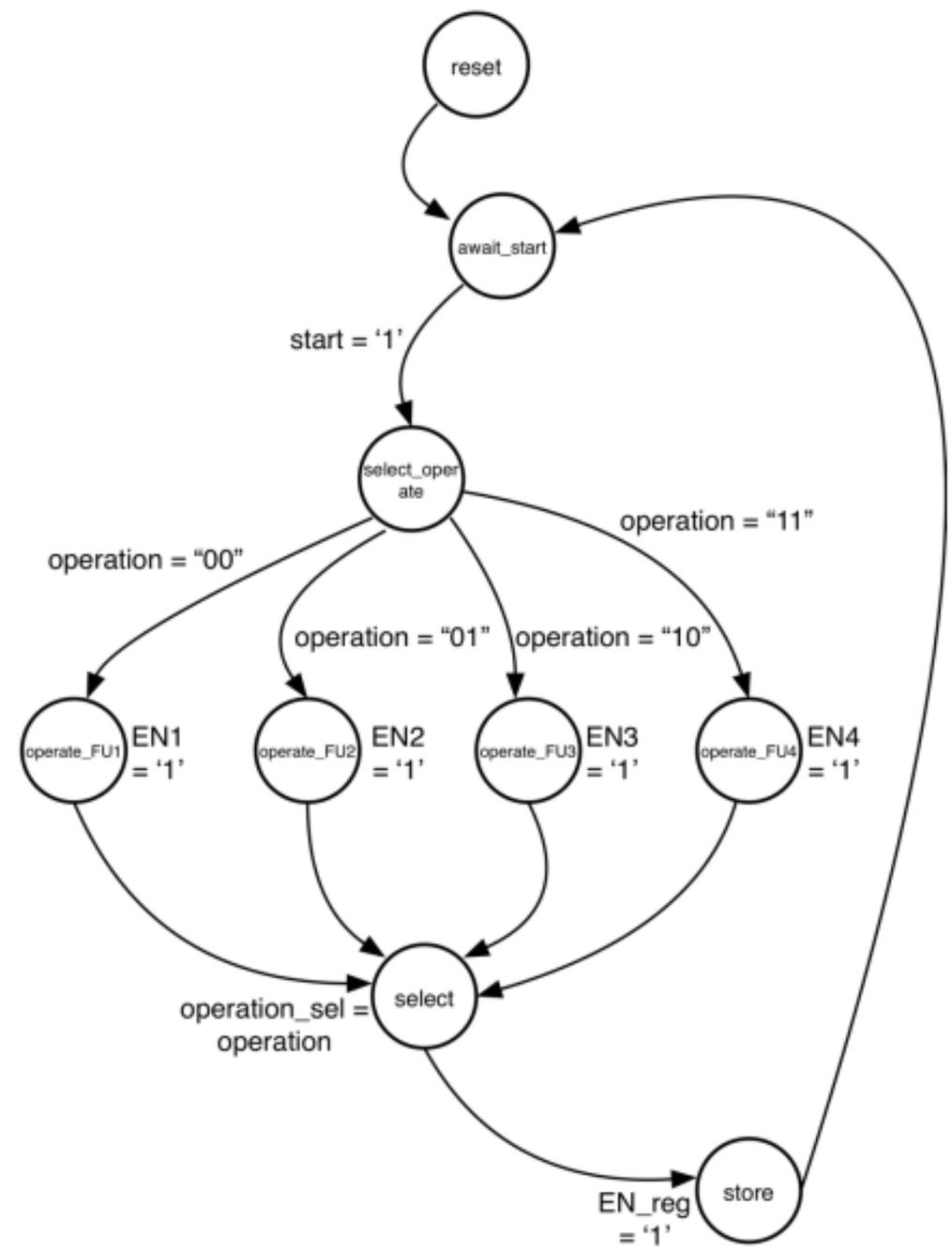
- Come from experience
- General guidelines for a good state machine
  - Control signals are fully and directly dictated by current state
  - Each state reached by a unique walk
  - Each state takes one cycle
  - By the end of the cycle we know which state to go to next
  - Only as many states as absolutely necessary
  - Have an escape state to go to with external interrupt

STATES FOR THE EXAMPLE

- Reset state, always have this, always a special case
- Await_start
- Operate_select
- Operate1, Operate2, Operate3, Operate4
- Select
- Store

STATE TRANSITION DIAGRAM

- Best visual representation
- Allows quick understanding of flow



STATE TABLE

- A good reference for control signal values
- Explicit and clear

Current state	Next state	Control signals
Reset	await_start	All nulls
await_start	select_operate if start='1'	All nulls
select_operate	Operate1 if operation='00'	All nulls
EN1, EN2, EN3, EN4	Select	Appropriate ENx
Select	Store	Appropriate sel
Store	await_start	EN_reg='1'

QUESTIONS ABOUT THE STATE MACHINE

- When in a state what is the next state
  - Can determine next state based on condition of status signals
- When in a state what are the outputs (control signals)
  - Preferably determined unconditionally
- Information should be clear from either state table or state diagram

THE THREE PROCESS APPROACH

- Systematic
- Yields predictable readable and optimally synthesizable state machines
- Divide state machine into three processes
  - The state update process (sequential)
  - The next state process (combinational)
  - The current output process (combinational)

DECLARATION

- ~~Declares a new enumeration~~
- ~~States are variables in the this new type~~
- ~~Synthesizers will translate into efficient registers~~
- ~~Enumeration allows great readability~~

```
Library IEEE;
Use ieee.std_logic_1164.all;

Entity fsm is
Port ( operation  : in std_logic_vector(1 downto 0);
        FU_done : in std_logic;
        start : in std_logic;

       EN1, EN2, EN3, EN4 : out std_logic;
       EN_reg : out std_logic;
       operation_sel : out std_logic_vector (1 downto 0);

      clk, reset : in std_logic;
End fsm;

Architecture behavioral of fsm is
Type state is (reset_state, await_start, operate_select, operate1, operate2,
operate3, operate4, select, store);
Signal current_state, next_state : state;
Begin
```

THE STATE UPDATE PROCESS

- ~~The only clocked process~~
- ~~When the new edge arrives, make whatever we decided was the next state be the current state~~
  - ~~Triggers a recalculation of next_state~~
- ~~There is a clear and explicit return to reset state upon an asynchronous reset~~
  - ~~Should come from global reset~~

```
Process(clk, reset)
Begin
 if reset = '1' then
  current_state <= reset_state;
 elseif clk'event and clk = '1' then
  current_state <= next_state;
 end if;
End process;
```

NEXT STATE PROCESS

- ~~A combinational process~~
  - ~~Double check sensitivity list~~
- ~~A single case structure~~
- ~~For every state determine next state~~
  - ~~Based on diagram~~
  - ~~Conditions derived from status bits only~~

```
Process (current_state, start, opeation)
Begin
 case current_state is
  when reset_state =>
   next_state <= await_state;
  when await_state =>
   if start = '1' then
    next_state <= operate_select;
   else
    next_state <= await_state;
   end;
  when operate_select =>
   if operation = "00" then
    next_state <= operate1;
   elsif operation = "01" then
    next_state <= operate2;
   elsif operation = "10" then
    next_state <= operate3;
   else
    next_state <= operate4;
   end;
...
  when operate1 =>
   next_state <= select;
  when operate2 =>
   next_state <= select;
  when operate3 =>
   next_state <= select;
  when operate4 =>
   next_state <= select;
  when select =>
   next_state <= store;
  when store =>
   next_state <= await_start;
  when others =>
   next_state <=reset_state;
 end case;
end process;
```

OUTPUT PROCESS

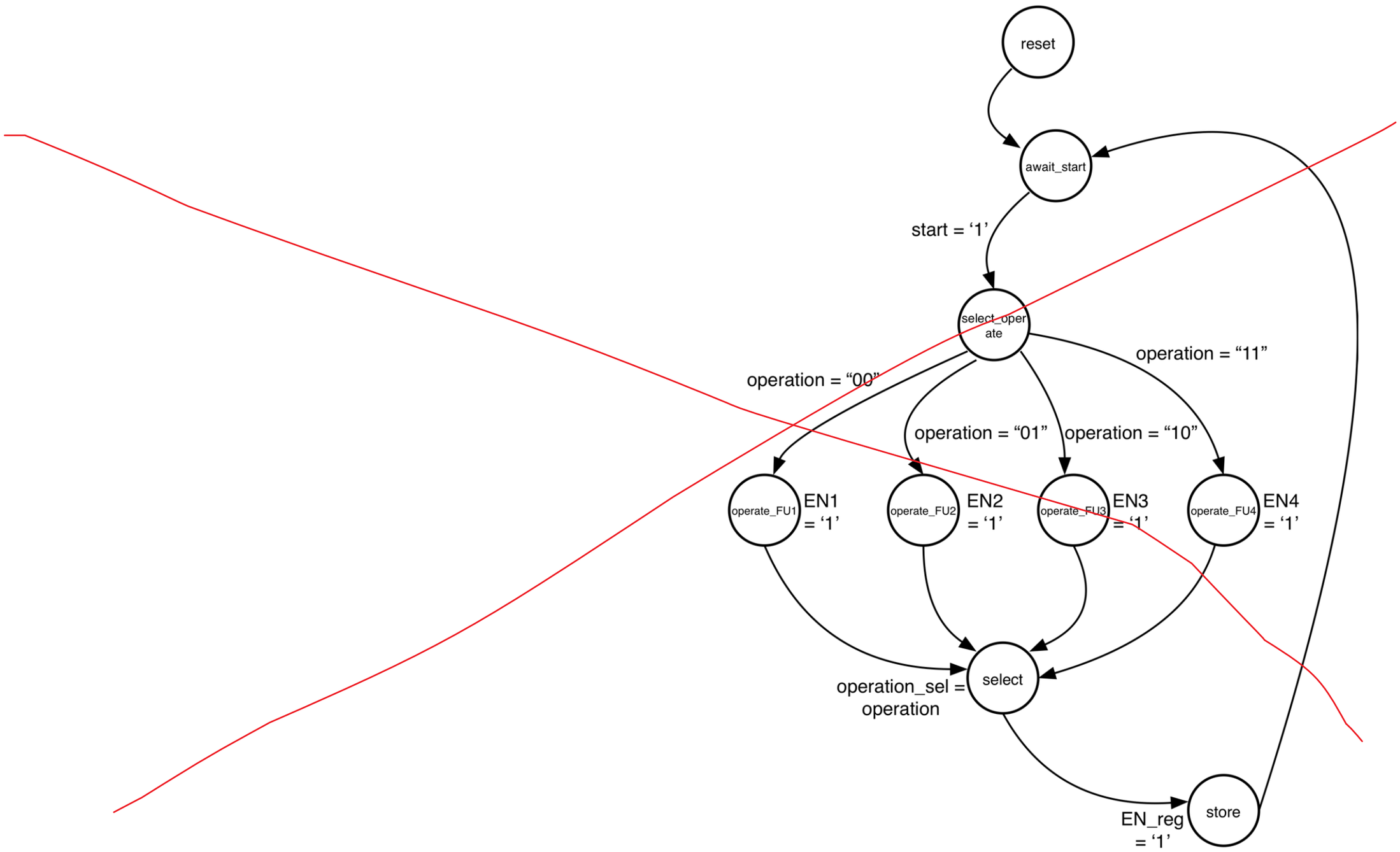
- ~~Combinational~~
- ~~Single case statement~~
- ~~For each state produce outputs~~
- ~~Outputs should be unconditional~~

```
Process(current_state, operation)
Begin
Case current_state is
When reset_state =>
EN1 <= '0'; EN2 <= '0'; EN3 <= '0'; EN4 <= '0';
EN_reg <= '0';
operation_sel <= "00";
When await_state =>
EN1 <= '0'; EN2 <= '0'; EN3 <= '0'; EN4 <= '0';
EN_reg <= '0';
operation_sel <= "00";
When operate_select =>
EN1 <= '0'; EN2 <= '0'; EN3 <= '0'; EN4 <= '0';
EN_reg <= '0';
operation_sel <= "00";
When operation1 =>
EN1 <= '1'; EN2 <= '0'; EN3 <= '0'; EN4 <= '0';
EN_reg <= '0';
operation_sel <= "00";
...
```

NOTES ON THE COMBINATIONAL PROCESSES

- Make sure all state possibilities are covered
- Use a when others statement and return to reset conditions
- Specify all outputs, even if trivial for all states
  - Otherwise there is a risk of implicit latching
- If you find you have to use conditions in the output process then maybe you have to split states

EXAMPLE ON SPLITTING STATES

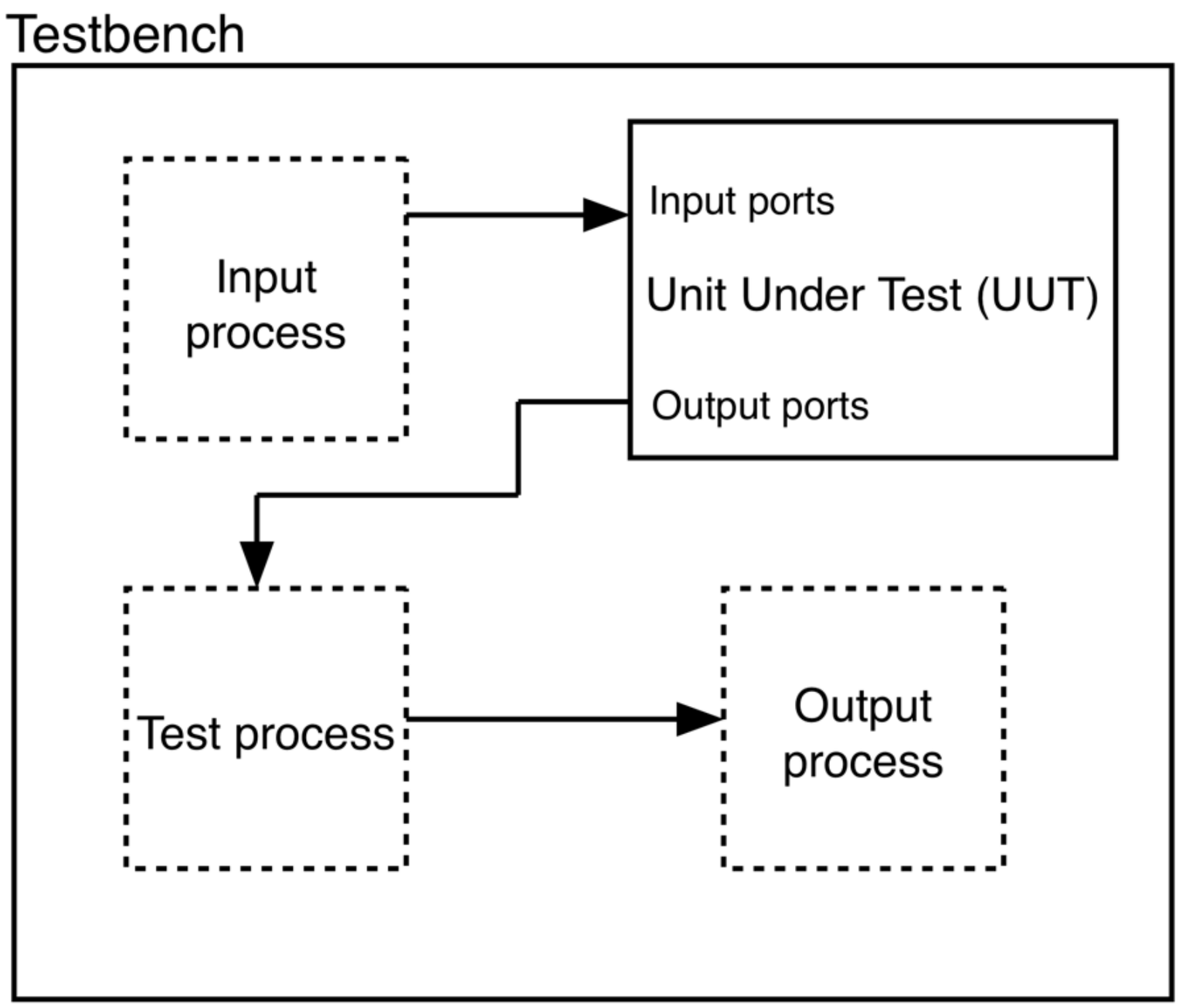


7B. TESTBENCHES, WAIT, ASSERT

DEFINITION

- A testbench is a special entity designed entirely for the purpose of testing a circuit
- The test bench entity is empty
  - It has neither ports nor generics
- The architecture of the testbench must instantiate at least one instance of the unit under test
- Most testbenches should not be subjected to synthesis
  - Can freely use unsynthesizable constructs

TESTBENCH ARCHITECTURE



THE UUT

- The Unit Under Test is the core of the test environment
- Every testbench must include at least one UUT
  - But there might multiple UUTs, although this can cause confusion
- The UUT is usually the wrapper of the highest level circuit
  - But is also often a sub circuit that needs to be tested independently
- When instantiating the UUT make sure to generic map all generic explicitly, do not rely on defaults
  - Promotes readability

INPUT PROCESS

- Generates test vectors
- Feeds vectors to UUT input ports
- Can use wait statements to create waveforms
- For most practical purposes use file I/Os to batch read tests

TEST PROCESS

- The process examines UUT outputs
  - Or alternatively internal signal values
- This is compared to a gold standard
- An error report is prepared and either output to the user, stored in a file, or dumped to console using assert statements

OUTPUT PROCESS

- The output of the UUT are either displayed or dumped in a way that allows the designer to examine them
  - Dump raw data, usually to an output file
    - Must involve a higher level of analysis
  - If the test process is more involved, dump an error indicator to the designer

HARDWARE TESTBENCHES

- Will be installed on hardware
  - Containing the core circuit implicitly within
- Typically done on FPGA
- Can dramatically speed up testing
- But design is much more restrictive than testbenches intended for simulation

INPUTS AND OUTPUTS

- VHDL supports two constructs for testbenches
  - Assertions
    - Output constructs
    - Ignored by synthesis tools
    - Generate error reports
  - Wait
    - Input constructs
    - Generate inputs
    - Complicated relation to synthesis

THE WAIT STATEMENT

- A process-bound statement
- Causes the process to pause, waiting for a set time or for a condition
- Wait statements look very non-synthesizable
  - And in many cases they are
  - But they also have a very fundamental relation to processes and sensitivity lists that has to be explored further

THE MANY FORMS OF WAITING

- Wait
  - Unconditional wait, causes process to freeze indefinitely
- Wait for *time*
  - Causes the process to wait for a specific amount of time
- Wait until *condition*
  - Condition written and evaluated as in if statements
- Wait on *signals*
  - Wait until an event has occurred on signals

WAIT-FOR

- Unquestionably non-synthesizable
- Process pauses for the amount of time specified
- The code snippet generates a clock with a period 200ns
  - But why does the process loop back??

```
process
begin
clk <= '1';
wait for 100ns;
clk <= '0';
wait for 100ns;
end process;
```

APERIODIC SIGNALS GENERATION USING WAIT

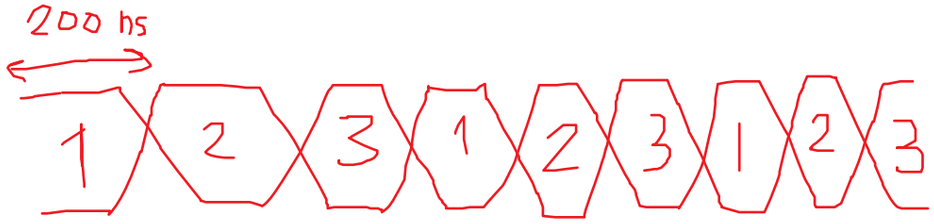
- Assume all processes **always** loop back
  - We will discover this is true even when using a sensitivity list
- The final wait statement is necessary to cause the process to cease looping and the signal to settle

```
process
begin
reset <= '1';
wait for 200ns;
reset <= '0';
wait;
end process;
```

STRANGE OCCURRENCES

- The bus Din will never assume the value 4
- It will be 1, 2, and 3 for 200ns each before repeating the cycle
- When the last statement is reached the value of Din is scheduled to be 4
- Immediately loop back and over write the schedule with value 1
- Start to understand when transactions become events

```
process
begin
Din <= conv_std_logic_vector(1,4);
wait for 200ns;
Din <= conv_std_logic_vector(2,4);
wait for 200ns;
Din <= conv_std_logic_vector(3,4);
wait for 200ns;
Din <= conv_std_logic_vector(4,4);
end process;
```



RESOLVING THE TRANSACTION

- The last wait statement updates the value of Din to 4
- It also freezes Din at this value
- Signal transactions become events at the first wait
- Until a wait is observed the signal assignment is just a transaction
  - ~~But how did this happen with a sensitivity list?~~

```
process
begin
Din <= conv_std_logic_vector(1,4);
wait for 200ns;
Din <= conv_std_logic_vector(2,4);
wait for 200ns;
Din <= conv_std_logic_vector(3,4);
wait for 200ns;
Din <= conv_std_logic_vector(4,4);
wait;
end process;
```

MORE EFFICIENT CODE FOR TIMELINE GENERATION

```
process
begin
Din <= conv_std_logic_vector(1,4) after 10ns;
        conv_std_logic_vector(2,4) after 100ns;
Q <= conv_std_logic_vector(3,4) after 100ns;
wait;
end process;
```

WAIT STATEMENTS AND SYNTHESIS

```
process
begin
wait until clk'event and clk='1';
Q <= D;
end process;
```

- Unconditional wait and wait for are not synthesizable
- Wait until is often synthesizable
- As shown above, it is equivalent to an if-else, creating DFF

WAIT STATEMENTS AND SYNTHESIS

```
process
begin
Q <= D;
Q1 <= D1;
Q2 <= D2;
wait until loop_again = '1';
end process;
```

- Wait until can also be used to create loops
- The synthesis status is similar to synthesis status of for loops

WAIT STATEMENTS AND SYNTHESIS

- This is a MUX
- The final wait statement acts exactly like the sensitivity list
  - Fully exchangeable with it
- Process with waits cannot have a sensitivity list
  - And vice versa

```
process
begin
if sel="00" then
Q <= A;
elsif sel="01" then
Q <= B;
elsif sel="01" then
Q <= C;
else
Q <= D;
end if;
wait on sel, A, B, C, D;
end process;
```

THE POWER OF WAIT STATEMENTS

- Wait statements can create complex conditions
- In this case the wait statement is replacing both the sensitivity list and an if statement

```
process
begin
if sel="00" then
Q <= A;
elsif sel="01" then
Q <= B;
elsif sel="01" then
Q <= C;
else
Q <= D;
end if;
wait on sel, A, B, C, D until clk'event and clk='1';
end process;
```

WAIT STATEMENTS AND SYNTHESIS

- An argument-free wait statement is never synthesizable
- Wait for statements are never synthesizable
- Wait until statements are often ok
- Wait on statements are usually fine
  - And exchangeable with sensitivity lists

WAIT AND TRANSACTIONS-EVENTS

- A signal assignment in a process is a transaction
- The transaction becomes an event at the first wait statement
- The wait statement must be explicit if there is no sensitivity list
- If there is a sensitivity list, the only place where events take place is the end of the process
  - The sensitivity list **is** a wait on statement at the end of the process
- All processes always loop indefinitely
  - Why did we not observe this with sensitivity lists?

ASSERTIONS

```
A <= B+C;
Aout <= A(15 downto 0);
-- B and C are 16 bits each
Assert A(16) ='1' report "overflow has occurred" severity error;
```

- Assertions can be used concurrently or sequentially
- They are ignored by synthesis (simulation construct)
- The assert statement checks a condition
  - If true, return the assertion report
  - Severity is an important distinction

SEQUENTIAL USE OF ASSERTS

```
process (clk)
begin
if clk'event and clk = '1' then
D1 <= D;
D2 <= D1;
D3 <= D2;
Dout <= D3;
end if;
assert report "Value of Dout is" & integer'image(conv_integer(Dout));
end process;
```

- Can be executed sequentially
- Note though that the value of Dout displayed is the old value

7C. FUNCTIONS AND PROCEDURES

STRIKINGLY SIMILAR BUT SUBSTANTIALLY DIFFERENT

- Functions and procedures are both types of sub-programs
- They hearken to programming
- They are declared, written, and used very similarly
- However, they differ substantially in how they return values
- They can be called in a process or outside, sequentially or concurrently

FUNCTIONS

- Functions are the simpler of the two sub-programs
- They are declared and described in the architecture declarations
  - If used in multiple files, use packages
- May have as many ports as needed
  - All ports are input without definition
- May (and must) explicitly return a single value
- Uses only variable assignment
- Variables and constants do not keep values between calls

EXAMPLE OF A FUNCTION DECLARATION

```
function quad_calc(a,b,c,x : integer)
    return std_logic_vector(16 downto 0) is
variable power1 : integer := 2;
variable power2 : integer := 1;
variable power3 : integer := 0;
variable return_value : integer;

begin
return_value := a*x**power1+b*x**power2+c*x**power3;
return conv_std_logic_vector(return_value,17);
end quad_calc;
```

IMPURE FUNCTIONS

- A pure function accepts inputs only through arguments
- Impacts the world only through return value
- An impure function typically uses or modifies shared variables

```
impure function raise_to_power(constant x : integer)
    return integer is
begin
return x**global_power;
end raise_to_power;
```

FUNCTIONS AND SYNTHESIS

- Functions encourage the use of programming features
  - Although they discourage the use of non-synthesizable constructs
- Functions can call each other, can be recursive, or can contain loops
  - They can also mix all the above
- This can lead to much worse synthesis results than a simple failure
- Rule of thumb: Any function call that reduces to a constant at synthesis time is perfectly fine to use

COMMON BUILT-IN FUNCTIONS

- Part of the std_logic definition in IEEE library
  - conv_integer(*std_logic vector*)
    - Returns an integer
  - conv_std_logic_vector(*integer, length*)
    - Returns a standard logic vector
  - conv_signed
  - conv_unsigned

PROCEDURES

- Procedures are more complicated than functions and differ in three significant ways:
  - They do not have return values and thus cannot be used in signal assignments
  - They can contain signal assignments
  - Their formal parameters have direction, allowing multiple return values
- Procedures are more akin to processes than functions

EXAMPLE OF A PROCEDURE

- When called, the parameters can be signals, variables, or constants
- Every parameter must have a direction and a class
- Inout direction parameters can cause confusion

```
procedure batch_calculate
( a, b, c : in integer;
  signal d, e, f : out integer;
  signal g : inout integer) is
begin
    d <= a*a;
    e <= b*b;
    f <= c*c;
    g <= g*g;
end batch_calculate;
```

OVERLOADING

- Both functions and procedures can be overloaded
- They can be declared multiple times significantly differently with the same name
- When called, the argument list allows the compiler to distinguish the overloading
  - Cannot overload with the same arguments/parameters

```
function raise_to_power(constant x,p : integer)
    return integer is
begin
return x**p;
end raise_to_power;

function raise_to_power(constant a,x,p : integer)
    return integer is
begin
return a*x**p;
end raise_to_power;
```

PRACTICAL USES OF OVERLOADING

- Overloading is often used to allow a function or procedure to be used with multiple data types
- Avoids the need to declare multiple distinct functions
  - Reduces the number of calls the designer has to remember

```
function accum(a: integer)
    return integer is
begin
return a+a;
end accum;

function accum(a : std_logic_vector(10 downto 0))
    return std_logic_vector is
begin
return a+a;
end accum;
```

7D. LOOPS, FILES

WHILE LOOP

```
process (Din)
variable loop_condition : integer;
begin
loop_condition := 0;
while loop_condition < 10 loop
if conv_integer(Din) = loop_condition then
 flag <= '1';
end if;
end loop;
end process;
```

FOR LOOP

```
process (Din)
begin
for loop_condition in 0 to 9 loop
if conv_integer(Din) = loop_condition then
 flag <= '1';
end if;
end loop;
end process;
```

NEXT AND EXIT IN A LOOP

```
process (Din)
begin
for loop_condition in 0 to 9 loop
if conv_integer(Din) = loop_condition then
 flag <= '1';
end if;
if conv_integer(Din) = 5 then
exit;
end if;
if conv_integer(Din) = 5 then
next;
end if;
end loop;
end process;
```

SYNTHESIS AND LOOPS

- Loops are synthesizable as long as they do not use wait for statements
- Loops encourage very bad coding
  - Especially with next and exit statements
- Can lead to successful but unpredictable synthesis

THE ROLE OF FILES

- Files are very powerful testing constructs
  - But they are only simulation constructs
- Using files requires two packages
  - std_textio
    - Defines two data types: file and line
    - Four functions: Read, write, readline, and write-in
  - std_logic_textio
    - Overloads std_textio functions to accept std_logic types

DECLARING AND USING FILES

- Notice the mode while opening files
- Reading and writing is not immediately clear

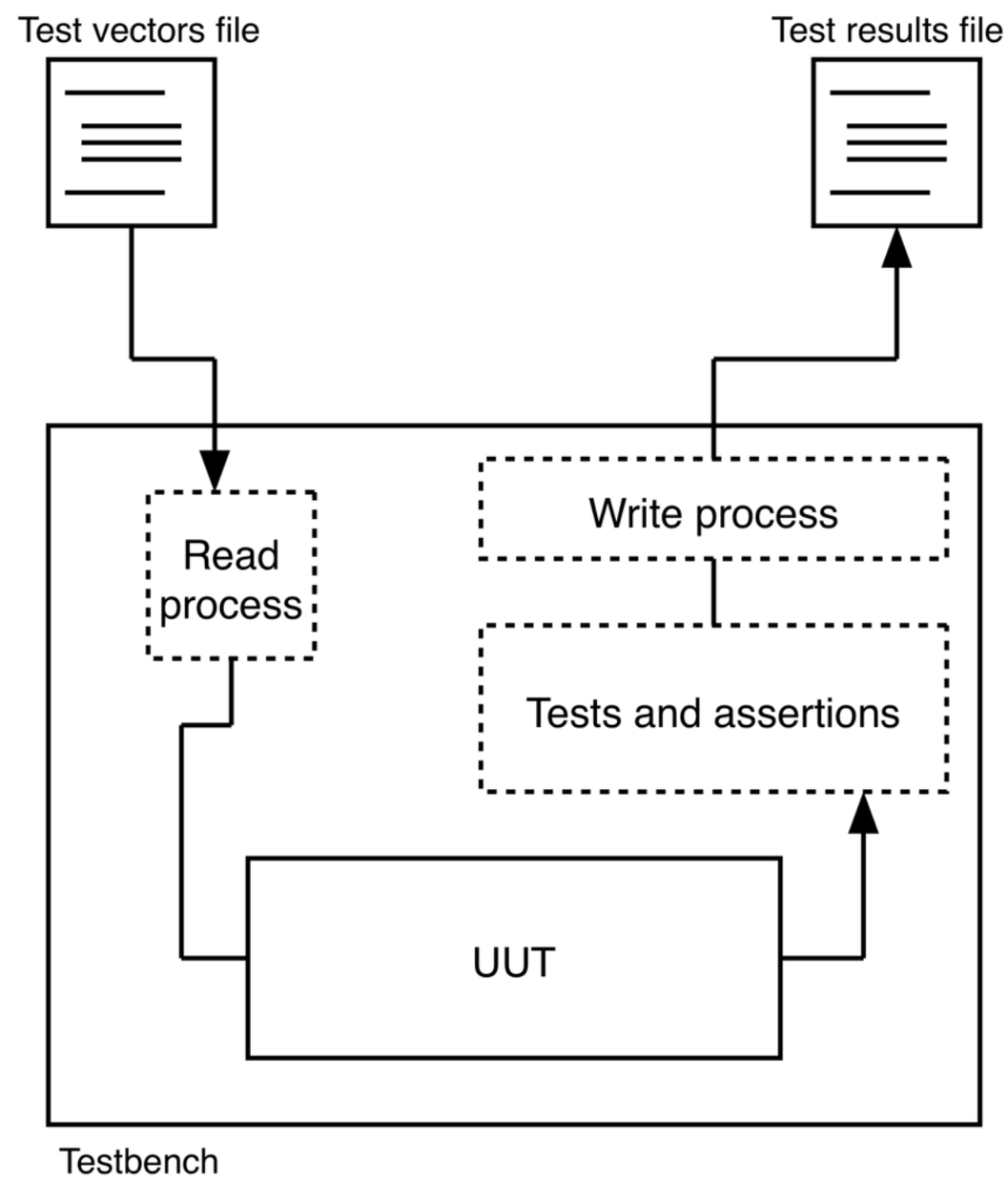
```
architecture behavioral of fileio is
file infile : text open read_mode is “c:\read_file.txt”;
file outfile : text open write_mode is “c:\write_file.txt”;
signal read_word, write_word : std_logic_vector(16-1 downto 0);
begin
write_word <= conv_std_logic_vector(21,16);
write_process: process
variable output_line : line;
begin
    write(output_line, write_word);
    writeline(outfile, output_line);
    wait;
end process;
read_process: process
variable input_line : line;
begin
    read(input_line, read_word);
    readline(infile, input_line);
    wait;
end process;
end architecture;
```

SERIAL READ/WRITE

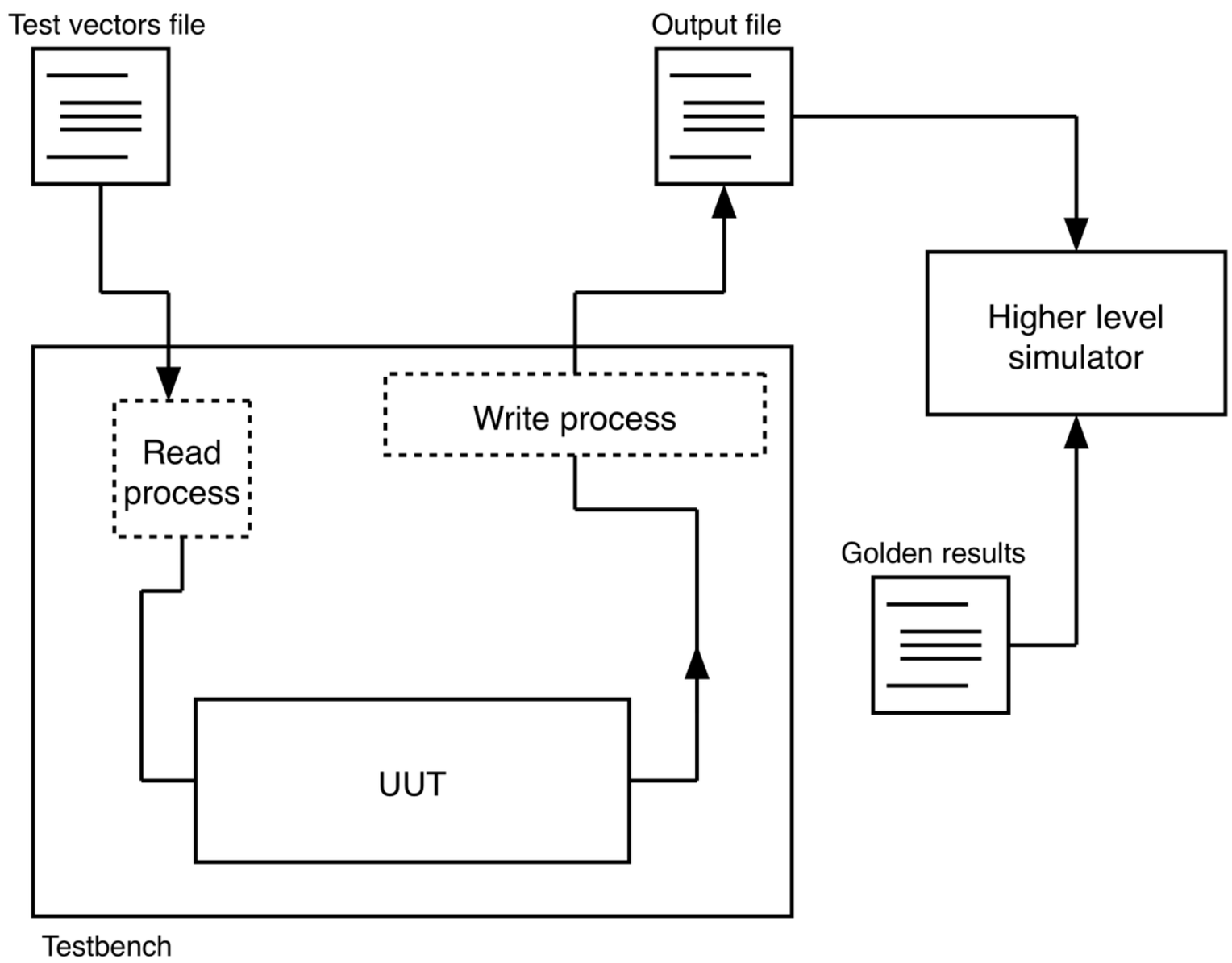
- Reading and writing are always serial
- Reading is not word by word
- Readline, a line is a physical line of data (variable length) in the file
  - User cannot directly observe the contents of the line
  - Function read is used to read words FIFO from the line
- Similarly write data to a line using the write function
  - Then use the writeline function to dump to file

FILES AND TESTING

- Batch reading of test vectors
- Batch writing of test results



RAW OUTPUT DUMPS



MEMORY LOADING

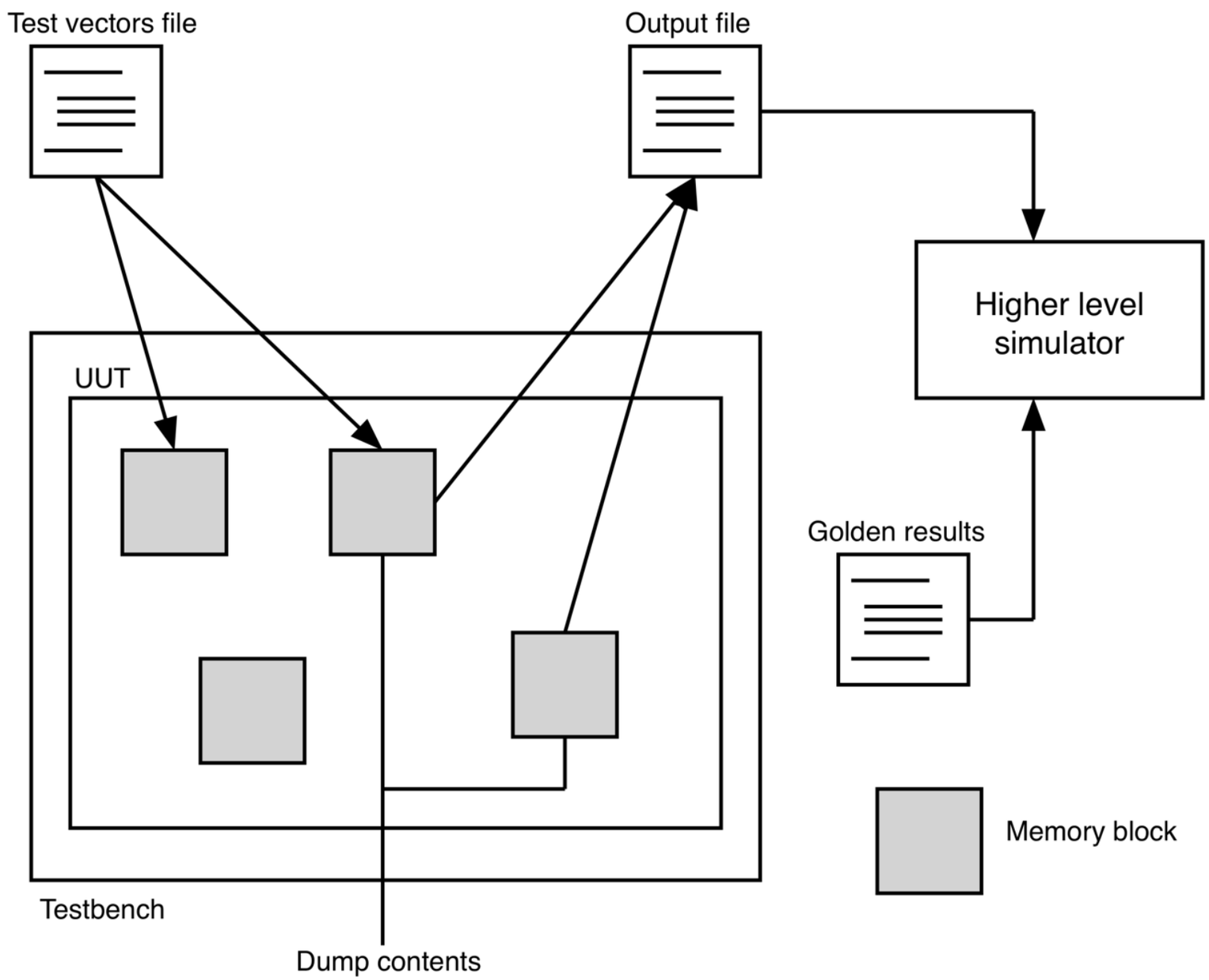
- This can pre-load memories with vectors in zero time
- Do not assume this would go beyond simulation

```
architecture behavioral of ram_block is
type ram_type is array((2**address_bits)-1 downto 0) of
    std_logic_vector(ram_word_width-1 downto 0);

impure function load_memory(load_file_name : in string) return ram_type is
    file ram_file : text open read_mode is load_file_name;
    variable read_line : line;
    variable ram_contents : ram_type;
begin
    for i in ram_type'range loop
        readline(ram_file, read_line);
        read(read_line, ram_contents(i));
    end loop;
    return ram_contents;
end function;

signal ram_memory : ram_type := load_memory("c:\load_file.dat");
begin
...
end architecture;
```

DATA DUMPING REQUIRES CONTROL SIGNALS



PROCESS TO DUMP DATA FROM MEMORY

- Even though there is a loop, it happens in zero simulation time

```
dump_mem : process(clk)
    variable out_line : line;
begin
    if clk'event and clk='1' then
        if dump_mem_contents='1' then
            dump_loop: for k in 0 to ram_type'range loop
                write(out_line, conv_integer(ram_memory(k)));
                writeline(output_file, out_line);
            end loop;
        end if;
    end if;
end process;
```

7E. PACKAGES

PACKAGES

- Packages are collections of data that is expected to be used across multiple entities. They include:
  - Functions and procedures
  - User-defined types and sub-types
  - Components
  - User-defined attributes
  - Constants
  - Attributes
  - Files

USING PACKAGES

- Once declared in a file in the work library, accessible for the rest of the design
- Can be included in any other file by using the *use* statement
- Packages are synthesizable if all their constituents are

PARTS OF A PACKAGE

- Declaration
- Body
- Both components can readily be rolled into the declaration
- If body is used function and procedure bodies must be in it

```
package my_synth_package is
constant bus_width : integer;
component full_add is
port(a, b, cin : in std_logic;
     s, cout : out std_logic);
function linear_combine(combine_vector : in std_logic_vector(2 downto 0));
     return std_logic;
end my_synth_package;
```

```
package body my_synth_package is
constant bus_width : integer := 16;
function linear_combine(combine_vector : in std_logic_vector(2 downto 0));
     return std_logic is
     begin
        return
        (combine_vector(0) xor combine_vector(0) xor combine_vector(0)
     end;
end my_synth_package;
```

USING PACKAGES

- Use statement used to access the package
- Use package.constituent can be used to access a particular part
  - Can avoid non-synthesizable parts
- Packages are very useful and their use should be encouraged

7F. GOOD PRACTICES

CORRECT, INCORRECT, SYNTHESIZABLE, AND GOOD

- Code with errors result in compilation failure
- Code that is not synthesizable causes synthesis errors
- Blatant bad practices yield synthesis warnings
- But some code passes synthesis smoothly
  - Only to yield strange and unpredictable hardware
- Good practices are general guidelines that yield good hardware
  - But some can be ignored if the designer is seasoned

GOOD PRACTICES 1

- Use a single clock
- If you use multiple clocks separate into clearly demarked sub-systems and carefully manage metastability
- Use only a single edge of the clock, preferably positive
- All registers must have a global asynchronous reset
- There should only be one global reset connected to all registers

GOOD PRACTICES 2

- Do not declare explicit latches
- Examine synthesis report for latch warnings and remove implicit latches
- If the process has a sensitivity list, fully populate it
- Processes should be used for registering and multiplexing only outside the testbench
- Avoid the use of loops except when results are predictable (shift registers, etc.)

GOOD PRACTICES 3

- Avoid the use of variables or mixing variables and signals
  - Unless their role is clear as in memories and counters
- Avoid the use of large register files, use memories
- Avoid pre-synthesized blocks to promote portability
- Do not use numbers, use constants and generics
- When using functions and procedures make sure they reduce to numeric values at synthesis time

GOOD PRACTICES 5

- The design must have a final wrapper
- The wrapper must contain only component instantiations
  - Remove glue logic
- Define all generics in the wrapper or give wrapper-level generics desirable default values
- All statements in the wrapper and below must be synthesizable and come from a library of synthesis constructs

GOOD PRACTICES 6

- Testbench can contain any syntax
- Testbench cannot be input to synthesis
  - Hardware testbench is an exception
- Use explicit mapping for ports, do not rely on order
- Always declare reasonable defaults for generics
  - But never rely on them and always generic-map
- Refer all generics to higher level generics up to the wrapper level

GOOD PRACTICES 7

- Avoid using non-integer generics
- Constant, one, and zero strings should always be in terms of generics or use the others=>'0' syntax
- Avoid complicated conditional statements, especially in the generate statement
- Do not nest more than two generate statements
- Enables are often not useful in registers, but if necessary make them synchronous

GOOD PRACTICES 8

- Processes should either contain registering or be combinational
- Better separate DSP into concurrent statements
- State machines should use the three process approach with user-defined state type
- Make sure all states in the state machine cannot be split

GOOD PRACTICES 9

- Always declare std_logic_1164 and use the standard logic type, never use the bit type
- Use the std_logic_arith package combined with either std_logic_unsigned or std_logic_signed (default to the latter)
- Do not use exponentiation or division unless calculating a constant
- Reserve wait statements for the test bench
- Use packages often and separate them into synthesizable and non-synthesizable packages
- Attributes that accept or return times cannot be used under wrapper